Task Graph Scheduling on Heterogeneous Platforms

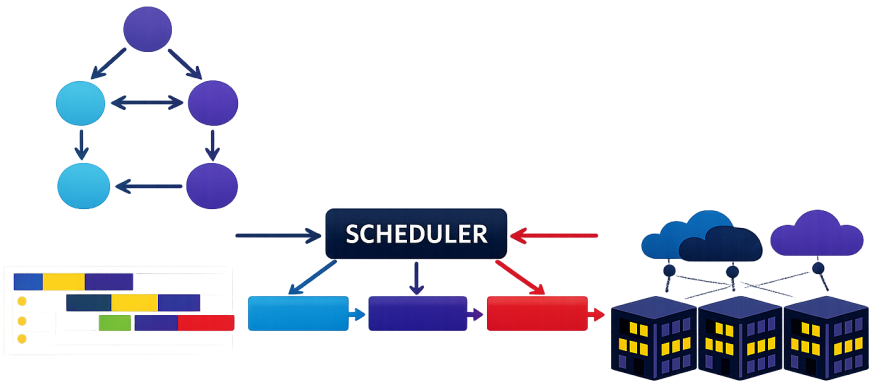
A Fast Sequential Implementation of the HEFT Algorithm

Group 3: Moncef AIT BELKACEM, Jingran CHEN, Niccolò TOZZI

April 2, 2026

Abstract

This report presents our work on task graph scheduling with precedence constraints on heterogeneous computing platforms. We focus on the Heterogeneous Earliest Finish Time (HEFT) algorithm, a static list-scheduling heuristic proposed by Topcuoglu *et al.* [1], and describe our sequential C++ implementation. We first introduce the problem and its relevance in modern high-performance and cloud computing, then review the state of the art and justify our choice of HEFT as a baseline heuristic in the bounded-number-of-processors (BNP) setting. We provide implementation details, correctness and performance results for the sequential version, and conclude with limitations and perspectives.



Contents

1 Introduction: Task Scheduling is hard **3**

2 Problem Statement and Context **3**
2.1 Formal problem definition: a neat mathematical representation . . . 3
2.2 Heterogeneous computing environment and BNP class . . . 5

3 Related Work and Bibliography **5**
3.1 Taxonomy of DAG scheduling algorithms . . . 5
3.2 Other baselines and metrics: reliable criteria . . . 6

4 A Fast Algorithm: HEFT **6**
4.1 Algorithm classification . . . 6
4.2 Two-phase structure . . . 7
4.3 Justification of the choice: HEFT is a practical baseline . . . 7

5 Input Model and Data Sets **8**
5.1 General heterogeneous case: consistency with the reference . . . 8
5.2 Simplified homogeneous case . . . 8

6 Sequential C++ Implementation **9**
6.1 Language and libraries: lightweight and robust . . . 9
6.2 Code organization . . . 9
6.3 Optimized Insertion-Based Policy . . . 10
6.4 Parallelization of EST computations . . . 10

7 Results **10**
7.1 Correctness on the reference instance . . . 10
7.2 Cross-validation with an open-source Python implementation . . . 11
7.3 Comparison against an optimal solver . . . 11
7.4 Performance and scalability . . . 11
7.4.1 Speedup w.r.t. an independent python implementation . . . 11
7.4.2 Performances in the homogeneous scenario . . . 12
7.4.3 Large-scale experiment . . . 13
7.4.4 Parallelization experiments . . . 13

8 Cloud Deployment **13**
8.1 Preliminary experiments . . . 14
8.2 Cloud architecture . . . 14
8.3 Implementation details . . . 14

9 Discussion and Conclusion **14**
9.1 Discussion . . . 14
9.2 Conclusion and future work . . . 15

1 Introduction: Task Scheduling is hard

Modern applications in high-performance computing (HPC) and cloud environments often consist of multiple interdependent tasks that must be executed efficiently on a set of parallel machines. These tasks naturally form a *directed acyclic graph* (DAG) (Fig. 1), where vertices represent computational tasks and edges encode precedence and communication constraints.

The core scheduling objective in this context is to assign tasks to processors and to determine their start times such that precedence constraints are satisfied and the overall completion time, or *makespan*, is minimized. This family of scheduling problems is known to be NP-hard already for identical parallel machines with precedence constraints and no communication costs, and remains NP-hard in more general heterogeneous and communication-aware settings. As a consequence, practical solutions rely on heuristic and metaheuristic algorithms that trade exact optimality for tractable runtimes and good empirical performance.

Large-scale task graph optimization reduces the makespan and thus operating costs in cloud data centers, improves energy efficiency, and ultimately affects end-user quality of service for latency-sensitive workloads such as video streaming or large-scale data processing services.

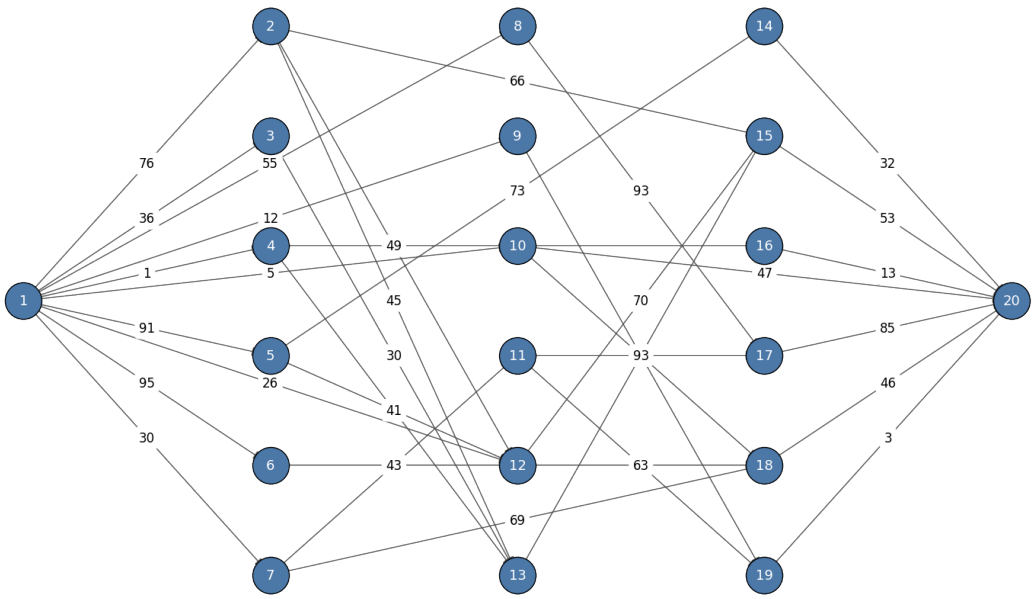


Figure 1: Example of task dependencies represented in a DAG. Nodes represent tasks, and directed edges represent dependencies. Edge weights represent communication cost.

2 Problem Statement and Context

2.1 Formal problem definition: a neat mathematical representation

We consider the static scheduling problem of an application represented by a directed acyclic graph (DAG)

$$G = (V, E),$$

where $V = \{t_1, \ldots, t_v\}$ is the set of $v = |V|$ tasks and $E \subseteq V \times V$ is the set of precedence edges. An edge $(t_i, t_j) \in E$ means that task t_j cannot start before task t_i has completed and the data produced by t_i has been made available to t_j. In HEFT, nodes represent tasks and edges represent inter-task data dependencies.

We consider a set of q heterogeneous processors

$$P = \{p_1, \ldots, p_q\},$$

assumed to form a fully connected platform.

The application and platform are characterized by the following quantities:

- A computation-cost matrix

$$W \in \mathbb{R}_+^{v \times q}, \qquad W_{i,m} = \text{execution time of task } t_i \text{ on processor } p_m.$$

- A communication-data matrix

$$\text{data} \in \mathbb{R}_+^{v \times v}, \qquad \text{data}_{i,j} = \begin{cases} \text{amount of data sent from } t_i \text{ to } t_j, & \text{if } (t_i, t_j) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

- A processor start-up vector

$$L \in \mathbb{R}_+^{q}, \qquad L_m = \text{communication start-up cost associated with processor } p_m.$$

- A bandwidth matrix

$$B \in \mathbb{R}_+^{q \times q}, \qquad B_{m,n} = \text{data transfer rate from processor } p_m \text{ to processor } p_n.$$

These are exactly the graph/platform attributes used by HEFT and CPOP.

For an edge $(t_i, t_j) \in E$, if t_i is scheduled on processor p_m and t_j on processor p_n, the communication cost is defined by

$$c_{i,j}^{m,n} = \begin{cases} 0, & \text{if } m = n, \\ L_m + \dfrac{\text{data}_{i,j}}{B_{m,n}}, & \text{if } m \neq n. \end{cases}$$

Thus, intra-processor communication is neglected, while inter-processor communication includes both a start-up latency and a transmission term.

A schedule assigns to each task t_i:

- a processor $\pi(t_i) \in P$,
- a start time $s(t_i) \geq 0$.

The finish time of t_i is

$$f(t_i) = s(t_i) + W_{i,\pi(t_i)}.$$

The schedule must satisfy:

1. **Precedence constraints:** for every $(t_i, t_j) \in E$,

$$s(t_j) \;\geq\; f(t_i) + c_{i,j}^{\pi(t_i),\,\pi(t_j)}.$$

   Equivalently,

$$s(t_j) \;\geq\; s(t_i) + W_{i,\pi(t_i)} + c_{i,j}^{\pi(t_i),\,\pi(t_j)}.$$

2. **Resource constraints:** if two distinct tasks $t_i \neq t_j$ are assigned to the same processor, then their execution intervals do not overlap:

$$\pi(t_i) = \pi(t_j) \implies [s(t_i), f(t_i)) \cap [s(t_j), f(t_j)) = \varnothing.$$

The objective is to minimize the makespan

$$C_{\max} = \max_{t_i \in V} f(t_i) = \max_{t_i \in V} \big(s(t_i) + W_{i,\pi(t_i)}\big).$$

Hence, the HEFT scheduling problem is to determine a processor assignment π and start times $s(\cdot)$ such that all precedence and resource constraints are satisfied and the makespan is minimized.

2.2 Heterogeneous computing environment and BNP class

In contrast to homogeneous systems, heterogeneous computing platforms comprise processors with diverse speeds and possibly different architectures (e.g. CPUs, GPUs, or specialized accelerators). Communication bandwidths between processors may also vary, and communication startup costs are not negligible, so both computation and communication heterogeneity must be taken into account when designing scheduling algorithms.

In the DAG-scheduling literature, many static list-scheduling heuristics are grouped into the *Bounded Number of Processors* (BNP) class, which targets a fixed, finite set of processors and assumes a fully connected processor interconnect.[1] BNP algorithms typically operate directly on the DAG (no prior clustering), construct a global priority list of tasks, and then map tasks to processors in priority order using some earliest-start/earliest-finish rule.[2] Our setting follows this BNP-style model: we schedule a DAG on a fixed set of processors, treat the interconnect as fully connected, and focus on list-based heuristics rather than clustering or task-duplication strategies.

Concretely, in our work:

- The number of processors m is fixed and finite.
- The processor network is treated as fully connected (any processor can communicate with any other, with possibly different bandwidths).
- No task duplication is considered in our baseline HEFT implementation.
- We do not perform clustering of tasks into super-nodes; we schedule the original DAG directly.

Within this BNP context, deterministic list-scheduling heuristics (such as HLFET, ETF, HEFT, CPOP, DCP) are particularly attractive because they offer polynomial-time algorithms, good empirical schedule quality, and deterministic outputs. Metaheuristics (e.g. genetic algorithms, ant colony optimization, simulated annealing) can sometimes yield slightly better solutions but at significantly higher computational cost and with more hyper-parameters to tune.

3 Related Work and Bibliography

3.1 Taxonomy of DAG scheduling algorithms

The literature on DAG scheduling for parallel machines with precedence constraints is extensive. A high-level taxonomy (see, e.g., Popa *et al.* [2]) distinguishes several classes based on the underlying model and scheduling strategy:

- LNP (Limited Number of Processors),
- BNP (Bounded Number of Processors),
- UNC (Unbounded Number of Clusters),
- TBD (Task Duplication-based algorithms),
- ANP (Arbitrary Network of Processors).

Within the BNP setting, scheduling approaches can be further categorized as:

- **Heuristics**, including:
  - List scheduling (HLFET, CP, ETF, HEFT, CPOP, DCP, ...),
  - Clustering-based methods (DSC, CASS-II, EZ, ...),

  - Task-duplication approaches (CPFD, TANH, DLS, ...).

- **Metaheuristics**, such as:
  - Genetic algorithms (GATS, HBGADS, ...),
  - Ant colony optimization (ACO-DAG, PACO, ...),
  - Particle swarm optimization (PSO-S, BPSO, ...),
  - Simulated annealing (SA-DAG, MASA, ...).

Our project uses HEFT as a high-quality deterministic list scheduler, commonly regarded as a strong baseline for heterogeneous DAG scheduling because of its simple structure, intuitive cost function, and ease of implementation.

3.2 Other baselines and metrics: reliable criteria

Several metrics are standard for evaluating scheduling algorithms:

- **Schedule Length Ratio (SLR)**:

$$\mathrm{SLR} = \frac{\text{makespan}}{\text{critical path length}},$$

  where the critical path length is usually computed assuming execution on the fastest processor(s). An SLR of 1.0 corresponds to an optimal schedule.

- **Speedup**:

$$\mathrm{Speedup} = \frac{\text{sequential execution time}}{\text{parallel makespan}}.$$

  Ideal speedup equals the number of processors but is rarely attainable due to precedence and communication overhead.

4 A Fast Algorithm: HEFT

4.1 Algorithm classification

HEFT can be classified as follows:

- **Static, offline**: it assumes the DAG structure, task computation times and processor communication costs are known in advance and do not change during scheduling.

- **Heuristic, greedy**: it uses a priority-based greedy rule rather than exploring the entire search space or optimizing an exact objective.

- **List scheduling**: it constructs a global priority list of tasks and then assigns tasks to processors in that order.

These properties make HEFT especially suitable for HPC and cloud workloads where performance profiles and communication characteristics can be estimated offline or derived from profiling.

4.2 Two-phase structure

HEFT is a two-phase list-scheduling heuristic that computes a static priority (the *upward rank*) for each task, based on average computation and communication costs over all processors, and schedules tasks in decreasing order of this rank, assigning each task to the processor that yields the earliest finish time.

1. **Task ranking (priority computation)**:

   For each task i, HEFT computes an *upward rank* $\mathrm{rank}_u(i)$ defined recursively as

   $$\mathrm{rank}_u(i) = \bar{w}_i + \max_{j \in \mathrm{succ}(i)} \left(\bar{c}_{ij} + \mathrm{rank}_u(j)\right),$$

   where $\bar{w}_i$ is the average execution time of task i over all processors, $\mathrm{succ}(i)$ denotes the set of immediate successors of i, and $\bar{c}_{ij}$ is the average communication cost from i to j across all processor pairs. For exit tasks, $\mathrm{rank}_u(i) = \bar{w}_i$.

   Intuitively, this rank approximates the remaining cost along the longest path from task i to the exit of the DAG. Tasks with higher upward rank feed into heavier downstream computation and thus receive higher scheduling priority.

2. **Processor selection (list scheduling)**:

   Tasks are sorted in nonincreasing order of rank_u. For each task i in that order, HEFT considers all processors $m \in \{1, \ldots, p\}$ and computes:

   - The earliest start time $\mathrm{EST}(i, m)$, taking into account:
     - Completion times of all predecessors of i,
     - Communication delays if predecessors are scheduled on different processors,
     - Existing tasks already scheduled on processor m.
   - The earliest finish time $\mathrm{EFT}(i, m) = \mathrm{EST}(i, m) + W_{i,m}$.

   HEFT then assigns task i to the processor $m^\star$ that minimizes $\mathrm{EFT}(i, m)$.

   An important implementation detail is *insertion-based scheduling*: the earliest start time and end time for the task i on processor m's current schedule is the first gap in the schedule that respects the provided precedence and communication constraints. Direct predecessor tasks are already scheduled since the uprank is a topological ordering of tasks. This increases processor utilization and can reduce the makespan significantly.

The theoretical overall complexity for this model is $O(v^2 \cdot p)$[3] where $v = |V|$ is the number of nodes and p is the number of processors, making HEFT scalable to large DAGs.

4.3 Justification of the choice: HEFT is a practical baseline

We chose HEFT as our primary algorithm for several reasons:

- **Explicit support for heterogeneity**: It models processor-dependent execution times and heterogeneous inter-processor communication rates, matching realistic cloud and HPC platforms.
- **Strong empirical performance**: HEFT is widely reported as one of the best-performing list schedulers across diverse DAG shapes and heterogeneity settings, achieving near-optimal SLR in benchmarks.[1]
- **Reasonable complexity**: Its $O(v^2 \cdot p)$ worst-case time complexity allows scheduling very large DAGs (up to tens of thousands of tasks) in seconds on a single machine, as confirmed by our experiments.

- **Deterministic and reproducible**: For a given instance, HEFT produces the same schedule, which is advantageous for debugging, performance regression testing, and scientific reproducibility.

On the downside, HEFT remains a greedy heuristic and may therefore produce suboptimal schedules, especially when, in a real setting, performance estimates are inaccurate. It also assumes static task graphs and does not natively handle faults, dynamic arrivals, or time-varying communication characteristics.

According to the paper "Comparing Task Graph Scheduling Algorithms: An Adversarial Approach," [4] while the HEFT algorithm performs well on traditional benchmark datasets, adversarial analysis reveals significant structural vulnerabilities. The authors discovered adversarial problem instances where HEFT performs poorly compared to even the simplest baseline algorithms.

5 Input Model and Data Sets

5.1 General heterogeneous case: consistency with the reference

Our implementation supports a general input format that closely follows the model described in the original HEFT paper [1]. A configuration (task scheduling problem) is specified by:

- Number of processors q.
- Number of tasks v and the DAG $G = (V, E)$.
- A vector L_m of size q: communication startup cost for each processor m.
- A matrix data_{ij} of size $v \times v$: amount of data to be transmitted from task i to task j whenever there is a direct dependency.
- A matrix B_{mn} of size $q \times q$: bandwidth (data transfer rate) between processors m and n.
- A matrix $W_{j,m}$ of size $v \times q$: execution time of task j on processor m.

Communication costs are modeled as:

- If two communicating tasks i and j are scheduled on the *same* processor m, the communication cost is assumed negligible:
$$c_{ij}^{m,m} = 0.$$
- If task i is scheduled on processor m and task j on processor $n \neq m$, the communication cost is:
$$c_{ij}^{m,n} = \begin{cases} 0, & \text{if } i = j, \\ L_m + \dfrac{\text{data}_{ij}}{B_{mn}}, & \text{otherwise.} \end{cases}$$

Our general C++ implementation reads such configurations from a textual format.

5.2 Simplified homogeneous case

In addition to the general heterogeneous format, we implemented a simplified homogeneous case, corresponding to the task graphs produced by a Python script `graph_generator.py` provided in the project repository. The script can generate a random graph with a specified number of tasks and an optional maximum number of dependencies per task. If the maximum number of dependencies is not provided as input, it is calculated after the graph is generated. Tasks are added to the DAG by assigning random existing tasks as parents, while respecting the

dependency limit. The first task has no parent. Each generated graph is associated with a unique graph ID (seed), which allows the program to regenerate it if needed. The output graph is saved in JSON format.

In the homogeneous case:

- All processors are assumed to have identical speed and no communication startup time.
- We defined the data transfer rates between two different processors to be `1`.
- The `data` to be transfered to successor tasks is the `memory` field.

6 Sequential C++ Implementation

All source code for this project is available on the public GitHub repository:

https://github.com/MK8BK/HEFT_cpp

6.1 Language and libraries: lightweight and robust

We implemented HEFT as a sequential C++20 program to obtain an efficient and compact baseline implementation of the algorithm. C++ offers direct control over data layout and execution, which is useful when processing large DAG instances and constructing schedules with low overhead. The implementation relies primarily on the C++ Standard Library, in particular containers such as `std::vector` and `std::set`, together with standard algorithms.

For input parsing, we use `RapidJSON` (Tencent), which provides fast JSON serialization/deserialization for graph instances and scheduling parameters. Modern C++20 features, including `std::optional` and `std::tuple`, are used where they improve code clarity and type safety. The program is compiled with standard optimization flags such as `-O2 -Wall -Werror`.

6.2 Code organization

The codebase is organized around a small number of core components:

- `TaskSchedulingProblemConfig`: representation of a heterogeneous scheduling instance, including the task DAG, processor-dependent computation costs, and communication parameters.
- `HomogenousTaskSchedulingProblemConfig`: specialized representation for the homogeneous input model.
- `HeftAlgorithm`: implementation of the HEFT heuristic for the general heterogeneous setting. Its main responsibilities are:
  - computing average computation and communication costs,
  - computing upward ranks for all tasks,
  - selecting processors via insertion-based scheduling and constructing the final schedule.
- `HomogenousHeftAlgorithm`: HEFT implementation specialized to the homogeneous case.
- `Schedule`: data structure storing processor assignments together with task start and finish times; it also provides makespan computation.
- Input/output and validation helpers:
  - `fromJson` / `readTspc`: parsing routines for heterogeneous and homogeneous instances,

  - `checkSchedule` / `checkHomogenousSchedule`: verification utilities that check precedence and communication constraints.
  - `scheduleToString`: transforming into a json string.

- `main`: command-line entry point that
  - selects the input format (general vs. homogeneous),
  - enables or disables schedule verification and makespan computation.

- Preprocessor directives control other debugging and verbosity options (json input validation and parallel execution) at compile time.

The implementation is intentionally lightweight, with a total code size of roughly ~ 500 lines excluding external libraries.

The sequential C++ program is designed as a faithful implementation of the standard HEFT workflow: parse an instance, compute upward ranks, schedule tasks in non-increasing rank order using insertion-based processor selection, and optionally verify the resulting schedule. Experimental validation of correctness and performance is reported in Section 7.

6.3 Optimized Insertion-Based Policy

When computing the Earliest Start Time of a task on a processor, the direct way is to linearly traverse the processor schedule and look for gaps large enough to fit the task. An optimization that we did that allowed us to decrease the execution time by an order of magnitude is to cache the gaps of every processor and scan them during the EST computation. The gaps are updated on insertion. They are kept on an ordered set. The asymptotic worst-case complexity remains the same, though a constant factor less time is spent looking for gaps (constant depends on structure of graph: high communication time or no, high computation times or no ... etc).

6.4 Parallelization of EST computations

Although the main implementation is sequential, we also experimented with parallel execution. In particular, some independent computations, such as evaluating earliest start times for several processors during processor selection, can in principle be dispatched to separate worker threads. For this purpose, we integrated a lightweight task-based thread pool[5], which provides a fixed set of worker threads and a shared queue of submitted tasks. This design avoids repeatedly creating and destroying `std::thread` objects for small computations, and instead amortizes thread-management overhead across many fine-grained jobs. In practice, however, this approach usually does not lead to measurable performance improvements. The overhead associated with thread creation often outweighs the cost of the computations themselves unless the processor count is really large. If the number of processors is very big, this approach can approximately divide our running time by the number of available threads, which is a hardware defined limitation.

7 Results

7.1 Correctness on the reference instance

We first validated the implementation on the canonical example from Topcuoglu *et al.* [1], consisting of a 10-task DAG executed on 3 heterogeneous processors. On this instance, our implementation reproduces the reference HEFT schedule reported in the original paper, including both the task ordering and the task-to-processor assignments. The resulting makespan is exactly 80 time units.

In addition, we checked the generated schedules with the built-in `checkSchedule` function to confirm that all precedence constraints are respected and that communication delays are handled correctly.

7.2 Cross-validation with an open-source Python implementation

As an independent sanity check, we compared our C++ implementation against the open-source Python HEFT implementation by Mack [6]. We generated 1,000 random DAG instances with the following parameter ranges:

- number of tasks $n \in [20, 120]$,
- number of processors $p \in [2, 12]$,
- task durations $w \in [1, 200]$,
- edge weights $e \in [1, 100]$.

Both implementations were executed on the same instances, and we compared the resulting makespans. The maximum absolute difference was on the order of 10^{-3} to 10^{-2} time units (for example, approximately 0.005), which is negligible and consistent with floating-point rounding effects rather than a substantive algorithmic discrepancy. This experiment provides strong evidence that the C++ implementation matches an independent HEFT implementation across a broad set of randomly generated inputs.

7.3 Comparison against an optimal solver

To assess solution quality, we also compared HEFT against an optimal solver on 40 random DAG instances with $n \in [20, 40]$ tasks and $p \in [3, 6]$ processors. The optimal solver proved optimality in 38 out of 40 cases within a timeout of 440 seconds. Table 1 reports the aggregate results over those proven-optimal instances.

Table 1: Comparison of the achieved makespan of HEFT against the optimal solver on instances solved to optimality

Metric	**Value**
Avg. Ratio	1.0121
Max. Ratio	1.0424

These results indicate that, on the tested instances, HEFT remains close to optimal, with an average gap of roughly 1.2% and a worst observed gap of about 4.2%.

7.4 Performance and scalability

7.4.1 Speedup w.r.t. an independent python implementation

We evaluated the runtime of our sequential C++ HEFT implementation against an open-source Python implementation on synthetic DAG instances of increasing size. The benchmark was designed to isolate scheduling performance as much as possible: graph generation was excluded from the timed region, the Python measurement included only the call to `schedule_dag(...)`, while the C++ measurement included process launch, JSON parsing, scheduling, and stdout generation redirected to the null device. Each configuration was executed for 5 measured repetitions after 1 warmup run.

The experiments were performed on sparse DAGs with 4 processors. Table 2 reports the mean runtime for both implementations together with the speedup factor

$$\text{speedup}_{\text{py/cpp}}(n) = \frac{t_{\text{python}}(n)}{t_{\text{cpp}}(n)}.$$

Table 2: Runtime comparison between the sequential C++ HEFT implementation and the reference Python implementation.

#Tasks	#Edges	Python [s]	C++ [s]	Speedup py/cpp
10	17	0.00122	0.00773	0.16
100	197	0.10837	0.00753	14.38
1000	1997	17.52452	0.01400	1252.15
2000	3997	93.57567	0.02655	3524.49

The results show two distinct regimes. For the smallest instance ($n = 10$), Python is faster, which is expected because the C++ timing includes fixed overheads such as process startup and input parsing that dominate when the actual scheduling work is very small. For larger instances, the gap grows dramatically: at $n = 1000$, C++ is more than three orders of magnitude faster.

This trend indicates that our implementation scales much better with problem size than the Python reference implementation.

7.4.2 Performances in the homogeneous scenario

We also report the performance of our HEFT implementation for a homogeneous processor scenario. The maximum number of dependencies per node (denoted as *max_deps*) is set to 30, resulting in a sparse task graph. As illustrated in Figure 3, the execution time exhibits super-linear growth with respect to the number of tasks.

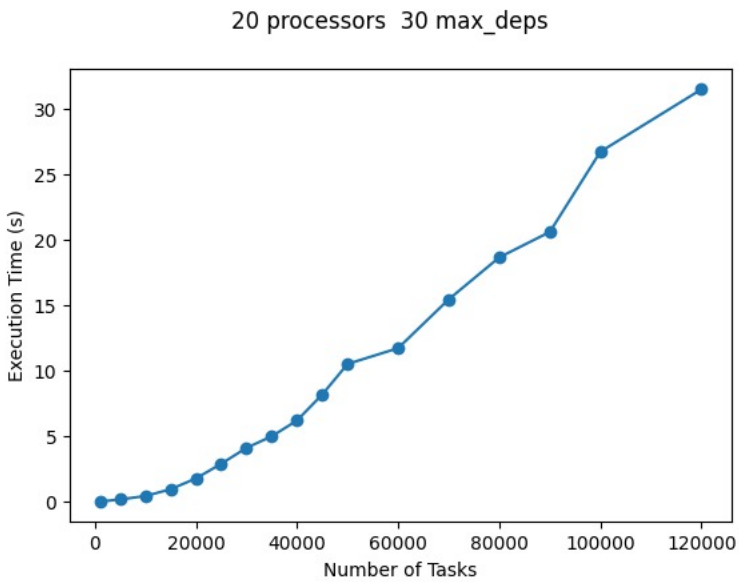


Figure 2: Execution time (in seconds) with respect to the number of tasks in a sparse graph for the homogeneous case. With non-zero communication costs.

7.4.3 Large-scale experiment

Finally, we evaluated the implementation on a very large DAG containing approximately 1.5×10^6 tasks.

The observed scheduling times are reported in Table 3. Even at this scale, the scheduler remained tractable, completing in less than 20 minutes for $q = 20$ processors and in 147 seconds for $q = 5$.

Number of processors q	Scheduling time
5	147 s
20	1202 s ($\approx$ 20 min)

Table 3: Scheduling time on the provided DAG with approximately 1.5×10^6 tasks.

As expected, the runtime increases with the number of processors considered during scheduling, since processor selection requires evaluating a larger set of candidates for each task. Nevertheless, the experiment shows that the optimized C++ implementation remains operational on very large instances.

7.4.4 Parallelization experiments

We tested the use of 2 threads on a 120k task graph with 40 processors, the computation time was $\approx$ 49 seconds compared to the non-parallelized version which clocked in at $\approx$ 70 seconds. On the 1.5 million task graph with 20 processors. We get the following results:

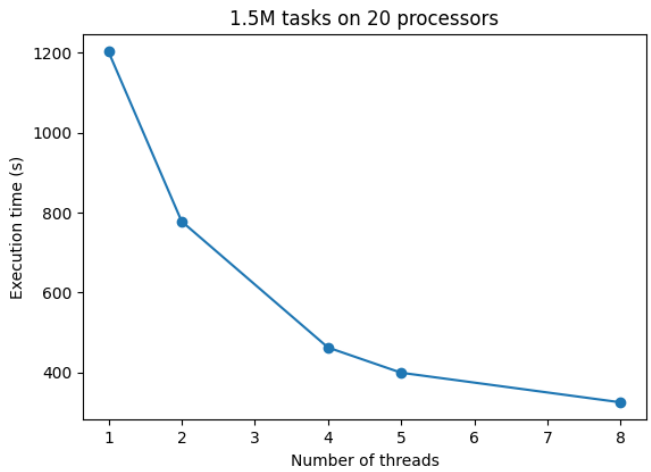


Figure 3: Thread-based parallelization results

These improvements in the asymptotic q case are bounded below by the number of physical threads available on the machine. We can notice Amdahl's law, considering the profiling information provided by `gprof` that states that $\approx$60% of the time is spent on the EST computation (which is parallelizable).

8 Cloud Deployment

One of the goals of the project was to deploy HEFT on a cloud platform in order to:

- Evaluate performance on realistic cloud resources,
- Explore parallelization opportunities beyond a single-node sequential implementation.

8.1 Preliminary experiments

For a given task, the EST is evaluated q times. These evaluations are mutually independent and do not alter the shared data structures, which makes them amenable to parallel execution. To exploit this property, a thread pool can be used to dispatch the EST computations among multiple threads. Nevertheless, this parallelization strategy is only beneficial under sufficiently large workloads. Indeed, the complexity remains $O(q \times v^2)$, so a substantial number of processors is needed for the parallel execution time to compensate for the associated overhead.
In practice, thread-pool creation and management introduce a non-negligible cost, which limits the effectiveness of this approach for moderate problem sizes.

8.2 Cloud architecture

AWS (Amazon Web Services) is the cloud platform used in our project[7], providing on-demand compute, storage, and orchestration services. Our architecture stores the input DAGs and the produced schedules in Amazon S3. The HEFT code is deployed as an AWS Lambda function, that is, a serverless function automatically executed in response to an event without managing servers directly. In this setting, each uploaded graph triggers one Lambda invocation, so one HEFT job is executed independently for each DAG. HEFT itself remains sequential for a single DAG, and AWS is therefore mainly used for coarse-grained parallelism by processing many DAGs concurrently through multiple Lambda invocations. This serverless design avoids the need for permanently running VM instances or containers, while still fitting naturally into a batch-style workflow. The resulting schedules are finally written back to S3, from which they can be downloaded and visualized during the analysis phase.

8.3 Implementation details

The existing C++ implementation was integrated into the cloud environment by packaging it as an AWS Lambda on a custom runtime[8]. In practice, the application was containerized with Docker in order to compile the executable in a Linux environment compatible with Lambda, rather than relying on the local toolchain. The build process produced a `bootstrap` executable, which was then zipped and deployed to the Lambda function.
Some issues remained unresolved at the end of the experimentation. Even after these architecture issues were corrected and the Lambda function could be invoked, the execution was unsuccessful. In particular, the invocation sometimes returned an HTTP status code `200`, which normally indicates that the request to Lambda was accepted, while the payload still contained `FunctionError: "Unhandled"` or runtime failures. In other words, the cloud service correctly received and launched the function, but the executable itself still aborted during execution. Therefore, although the deployment pipeline itself was progressively brought close to a working state, the C++ version remained only partially functional.

9 Discussion and Conclusion

9.1 Discussion

Our work confirms that HEFT is a practical and effective baseline for scheduling DAGs on heterogeneous platforms:

- The C++ implementation is capable of handling large graphs (tens of thousands of tasks) with moderate runtimes on a single machine.
- Verification against both the original reference example and an independent Python implementation provides strong evidence of correctness.

- The cloud deployment (once fully documented) illustrates how classical scheduling heuristics can be integrated into modern cloud-based workflows.

However, HEFT also exhibits well-known limitations:

- It is a greedy heuristic and cannot guarantee optimality.
- It relies on average execution and communication times, which may not fully capture runtime variability in cloud environments.
- It assumes a static DAG and does not adapt to dynamic workloads, failures, or changing resource availability.
- The classic formulation is not inherently parallel; parallelizing HEFT itself (beyond coarse-grained parallelism across instances) would require additional algorithmic work.

9.2 Conclusion and future work

In this project, we:

- Surveyed the state of the art in DAG scheduling for heterogeneous processors, positioning our work in the BNP / list-scheduling context.
- Implemented the HEFT algorithm in C++ for both general heterogeneous and simplified homogeneous models.
- Validated our implementation against the original HEFT paper and an open-source Python implementation.
- Evaluated performance on large synthetic DAGs and initiated a deployment on a cloud platform.

Possible directions for future work:

- Successfully deploying the C++ HEFT implementation on the cloud and completing large-scale experiments in a fully operational serverless setting.

References

[1] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. "Performance-effective and low-complexity task scheduling forheterogeneous computing". In: *Parallel and Distributed Systems, IEEE Transactions on* 13 (Apr. 2002), pp. 260–274. DOI: 10.1109/71.993206.

[2] Emilia Popa, Mauro Iacono, and Florin Pop. "Adapting MCP and HLFET Algorithms to Multiple Simultaneous Scheduling". In: *International Journal of Parallel Programming* 46 (June 2018). DOI: 10.1007/s10766-017-0516-z.

[3] Karan R. Shetti, Suhaib A. Fahmy, and Timo Bretschneider. "Optimization of the HEFT Algorithm for a CPU-GPU Environment". In: *2013 International Conference on Parallel and Distributed Computing, Applications and Technologies*. 2013, pp. 212–218. DOI: 10.1109/PDCAT.2013.40.

[4] Jared Coleman and Bhaskar Krishnamachari. *PISA: An Adversarial Approach To Comparing Task Graph Scheduling Algorithms*. 2024. arXiv: 2403.07120 [cs.DC]. URL: https://arxiv.org/abs/2403.07120 (visited on 04/01/2026).

[5] Andrew Lugowski. *task-thread-pool: Fast and lightweight thread pool for C++11 and newer*. https://github.com/alugowski/task-thread-pool. GitHub repository, accessed 2026-04-02. 2026.

[6] J. Mack. *HEFT: Heterogeneous Earliest Finish Time, Python Implementation*. GitHub repository. Update access date if needed. URL: https://github.com/mackncheesiest/heft (visited on 01/01/2026).

[7] Amazon Web Services. *Cloud Computing Services – Amazon Web Services (AWS)*. 2026. URL: https://aws.amazon.com/ (visited on 04/01/2026).

[8] AWS Labs. *aws-lambda-cpp: C++ implementation of the AWS Lambda runtime*. https://github.com/awslabs/aws-lambda-cpp. GitHub repository, accessed 2026-04-02. 2026.